

DOCUMENT 5 — QUALIFYAI DATABASE ARCHITECTURE & SECURITY MODEL

Document Type: Database Architecture & Security Model

Project: QualifyAI

Purpose: Complete database structure, entity relationships, tenant isolation, authentication mapping, access control, storage boundaries, and database security architecture.

1. Document Overview

This document defines the database architecture for **QualifyAI**, including:

- Identity architecture
- Organization and tenant model
- User roles
- Job and JD data
- Rubrics
- Questions
- Candidates
- Interviews
- Interview sessions
- Transcripts
- Evaluation results
- Communication metrics
- Proctoring events
- Reports
- Analytics
- Storage relationships
- Row-level security
- Data isolation
- Database access principles

The architecture follows the same type of database-first and security-first organization used in the reference database document, while all entities and workflows here are specific to QualifyAI. OpenHealth_Database_Architectur...

2. Database Design Objective

The QualifyAI database must guarantee that:

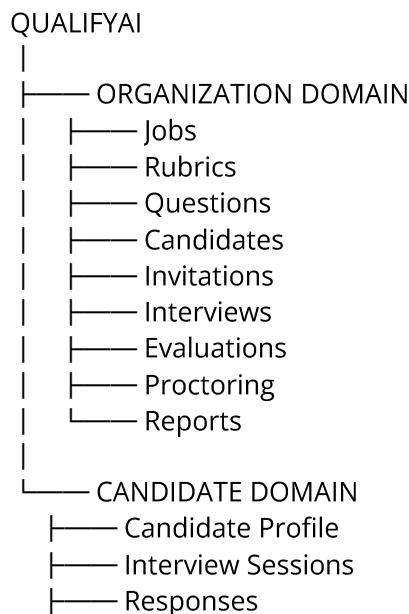
1. A user can access only authorized resources.
2. Recruiters can access only data belonging to their organization.
3. Candidates can access only their own candidate/interview information.
4. One organization cannot access another organization's jobs.
5. One organization cannot access another organization's candidates.
6. Interview transcripts remain restricted to authorized users.
7. Evaluation results remain tenant-scoped.
8. Proctoring information is accessible only to authorized parties.
9. Reports are associated with the correct interview and candidate.
10. AI-generated information cannot bypass authorization controls.
11. Frontend filtering is never treated as the primary security boundary.

The central principle is:

Database-level access control must enforce tenant and ownership boundaries.

3. QualifyAI Ownership Model

QualifyAI has two primary business ownership domains:





The fundamental ownership dimensions are:

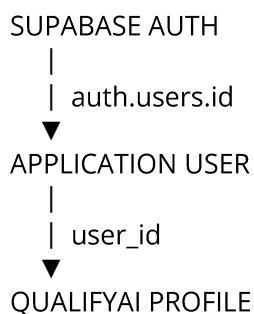
User identity → user_id
Organization data → organization_id
Candidate identity → candidate_id
Interview data → interview_id
Job data → job_id

A single generic user_id should not be used as the ownership key for every resource.

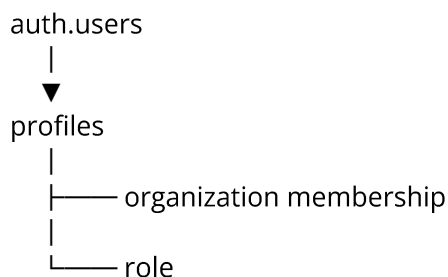
4. Identity Architecture

Supabase Auth is responsible for authentication.

The application database stores the corresponding application-level user information.



Conceptually:



The database should maintain a stable relationship between authentication identity and application identity.

5. User Role Model

QualifyAI should use a controlled role model.

Primary roles:

recruiter_admin
candidate

Additional role granularity may be introduced later if required by the product scope.

The initial role model should remain simple and explicit rather than creating unnecessary administrative roles.

6. Role Definitions

6.1 Recruiter Admin

A recruiter/admin associated with an organization can manage authorized organization resources, including:

- Jobs
- Job descriptions
- Rubrics
- Questions
- Candidates
- Interview invitations
- Interviews
- Candidate rankings
- Evaluation results
- Reports
- Organization-level analytics

Access must remain restricted to the recruiter's organization.

6.2 Candidate

A candidate can access:

- Their candidate profile
- Their invitations
- Their authorized interviews
- Their active interview session
- Their interview completion state
- Their diagnostic feedback/report where permitted

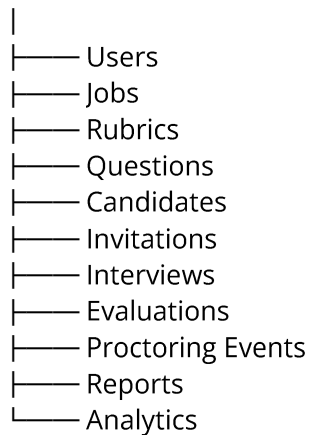
A candidate must not access:

- Other candidates
- Recruiter-only analytics
- Internal candidate rankings
- Other organizations
- Internal evaluation controls

7. Organization / Tenant Model

QualifyAI follows a multi-tenant SaaS architecture.

ORGANIZATION

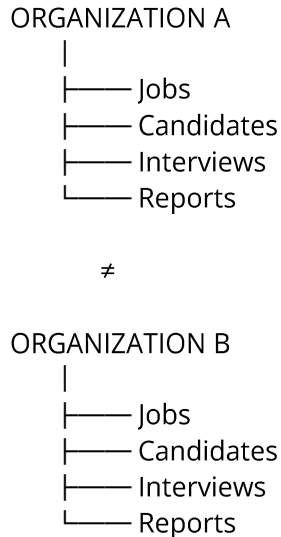


The organization is the primary tenant boundary.

Every organization-owned resource should be traceable back to its organization.

8. Multi-Tenant Isolation Principle

The core security rule is:



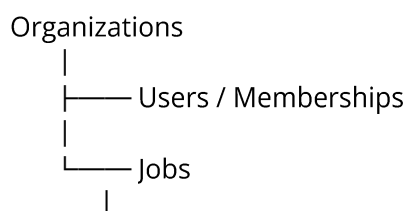
Organization A must never retrieve Organization B's data through:

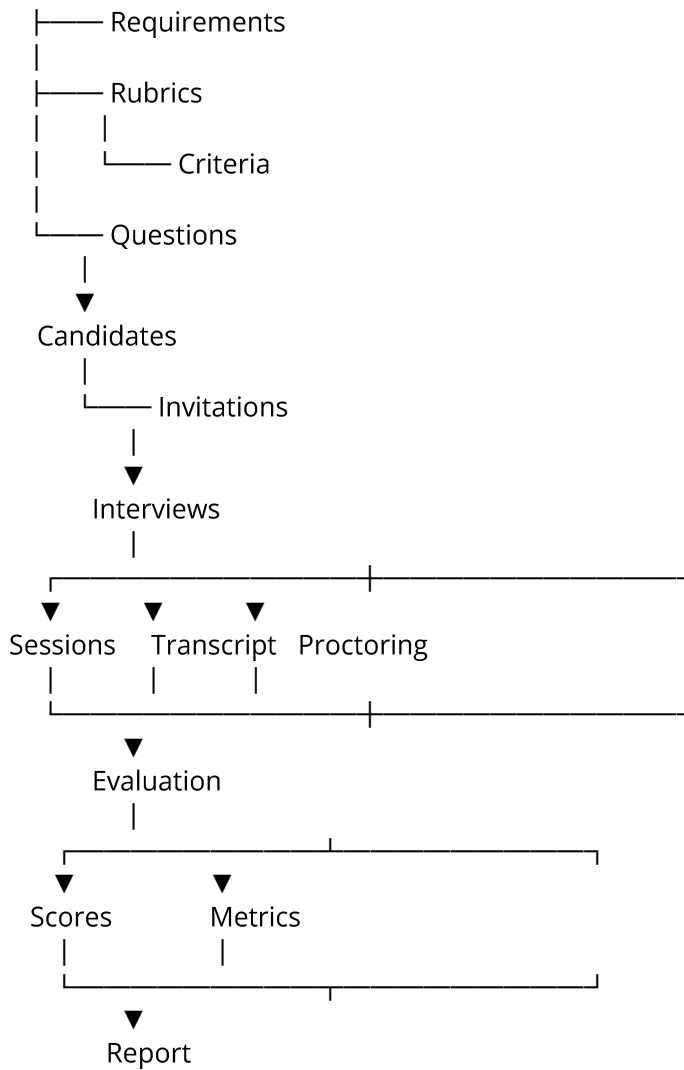
- API requests
- Direct database queries
- Client-side manipulation
- AI-generated requests
- Report URLs
- Interview session identifiers

Tenant isolation must be enforced server-side and, where applicable, through database-level policies.

9. Core Database Entity Model

The primary database model is:





10. Organizations Table

The organizations entity represents each recruiting company or institution using QualifyAI.

Conceptual fields:

Field	Purpose
id	Organization identifier
name	Organization name
slug	Organization-friendly identifier
status	Organization status
created_at	Creation timestamp
updated_at	Last update timestamp

All tenant-owned entities should ultimately reference this organization.

11. Profiles / Users Table

The application-level user profile represents authenticated users.

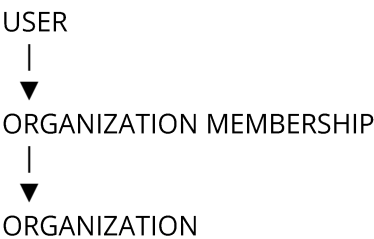
Conceptual fields:

Field	Purpose
id	Application user ID
email	User email
full_name	Display name
role	User role
created_at	Creation timestamp
updated_at	Last update timestamp

The authenticated identity should be mapped to the application profile.

12. Organization Membership

If the system needs users to belong to organizations through a separate relationship, the conceptual structure is:



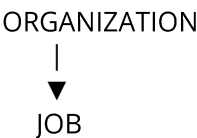
This allows future support for:

- Multiple recruiters
- Multiple organization users
- Organization-level access control
- Different recruiter permissions

The membership layer should become the authorization source for organization access.

13. Jobs Table

A job represents a recruitment position created by an organization.

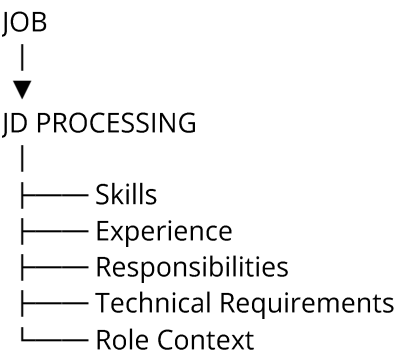


Conceptual fields:

Field	Purpose
id	Job identifier
organization_id	Tenant owner
title	Job title
description	Original JD content
status	Job status
created_by	Recruiter creator
created_at	Creation timestamp
updated_at	Last update

14. Job Requirements

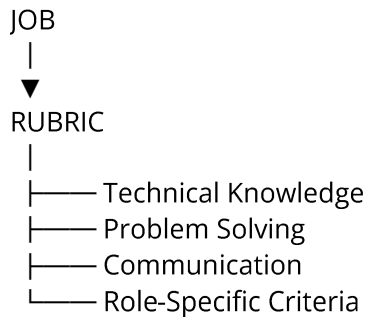
A processed JD can produce structured requirements.



The structured representation should support downstream rubric and question generation.

15. Rubric Architecture

Each job may have a role-specific rubric.

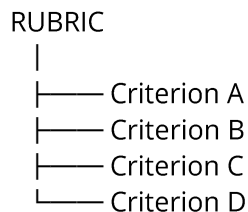


Conceptually:

Entity	Relationship
Organization	owns job
Job	has rubric
Rubric	has criteria
Criteria	contributes to evaluation

16. Rubric Criteria

Each rubric should be broken into evaluation criteria.



A criterion may conceptually contain:

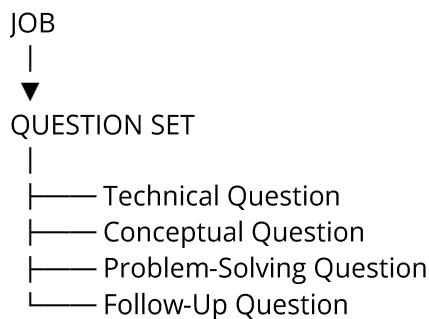
- Name
- Description

- Weight
- Expected competency
- Evaluation guidance

Weights should remain configurable.

17. Questions Architecture

Questions are associated with the relevant job/rubric context.

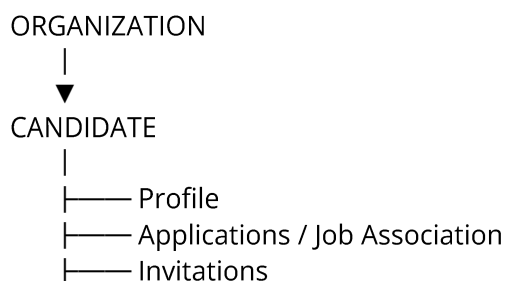


Questions should retain enough metadata to identify:

- Question purpose
 - Related skill
 - Difficulty
 - Rubric criterion
 - Question order/context
-

18. Candidate Architecture

Candidates belong to the recruitment process of an organization.



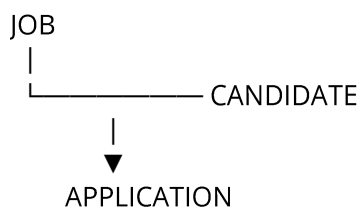
└─ Interviews

Candidate data must remain tenant-scoped.

A candidate record should not automatically grant access to all jobs or interviews.

19. Candidate-Job Relationship

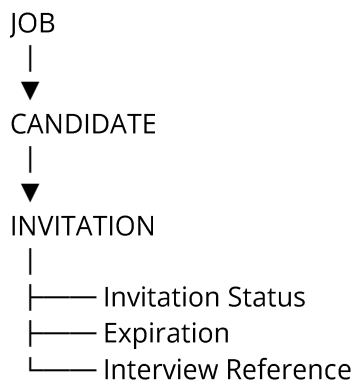
A candidate may be associated with a specific job.



This relationship allows the same candidate identity to potentially participate in different recruitment processes while preserving job-specific interview data.

20. Invitation Architecture

Recruiters can invite candidates to an interview.

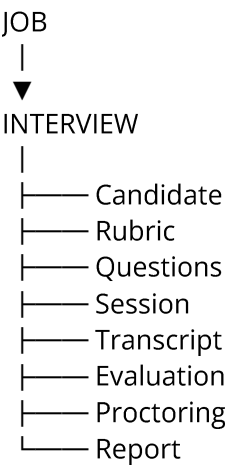


Possible states:

CREATED
SENT
OPENED

21. Interview Architecture

An interview is the central assessment entity.



Conceptual fields:

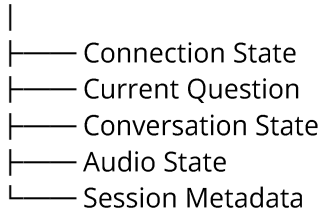
Field	Purpose
id	Interview ID
organization_id	Tenant
job_id	Associated job
candidate_id	Candidate
status	Interview status
started_at	Start time
completed_at	Completion time

22. Interview Session

The session represents the real-time execution of an interview.



INTERVIEW SESSION



An interview may have one primary active session according to the application's session policy.

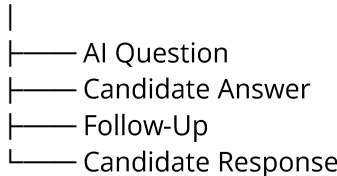
23. Transcript Architecture

The transcript stores the conversational record.

INTERVIEW



TRANSCRIPT



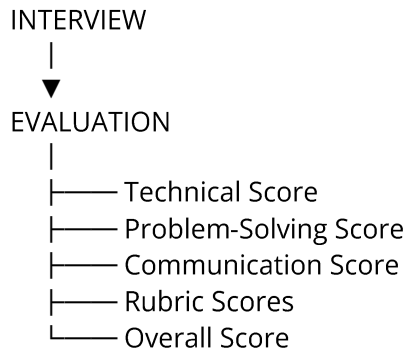
Each conversational segment should retain enough context to reconstruct the interview.

Conceptually:

Field	Purpose
interview_id	Interview reference
speaker	AI / candidate
content	Transcript text
sequence	Conversation order
timestamp	Event time

24. Evaluation Architecture

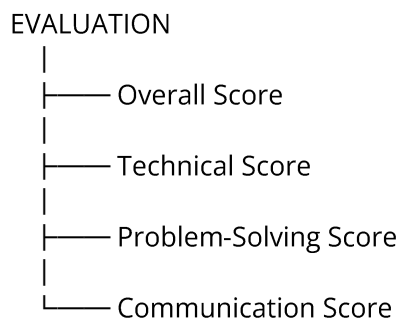
Evaluation is associated with the completed interview.



The evaluation should preserve the relationship between the score and the underlying interview.

25. Score Architecture

Scores should support both overall and criterion-level results.



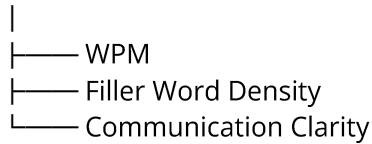
The documented product direction uses a **0-100 scoring model**.

The database should retain sufficient detail to explain how the final score was formed.

26. Communication Metrics

Communication-related data should be stored separately from technical scoring where practical.

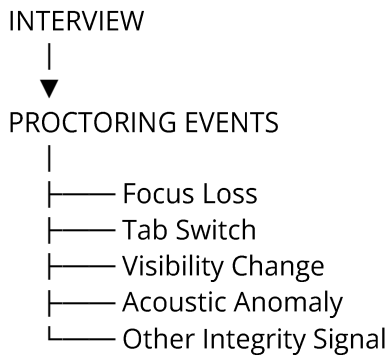




These metrics can then be surfaced in candidate feedback and recruiter analytics.

27. Proctoring Event Architecture

Proctoring events should be associated with the relevant interview.

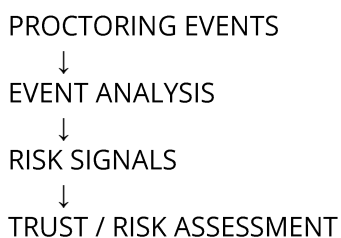


Each event should retain:

- Event type
 - Timestamp
 - Interview reference
 - Severity/risk information where applicable
 - Relevant metadata
-

28. Proctoring Risk Architecture

Individual events can contribute to an interview-level integrity assessment.



↓
PROCTORING SUMMARY

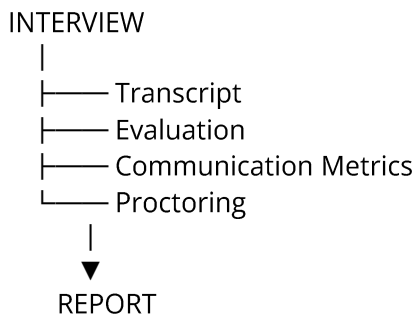
The system should distinguish between:

- Raw events
- Derived signals
- Final summary

This prevents the loss of the original evidence.

29. Report Architecture

Reports are generated from completed interview data.



A report can contain:

- Candidate summary
 - Overall score
 - Technical evaluation
 - Communication metrics
 - Strengths
 - Improvement areas
 - Interview summary
 - Integrity/proctoring summary
-

30. Candidate Diagnostic Report

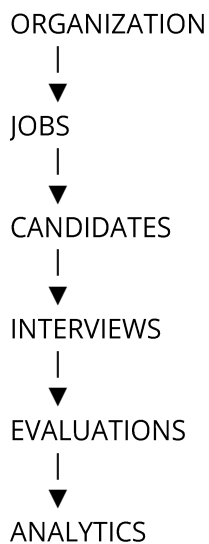
The candidate-facing report should expose only information permitted by the product's candidate-report policy.



Candidate access must be checked independently from recruiter access.

31. Recruiter Analytics Data

Analytics should be derived from organization-scoped assessment data.

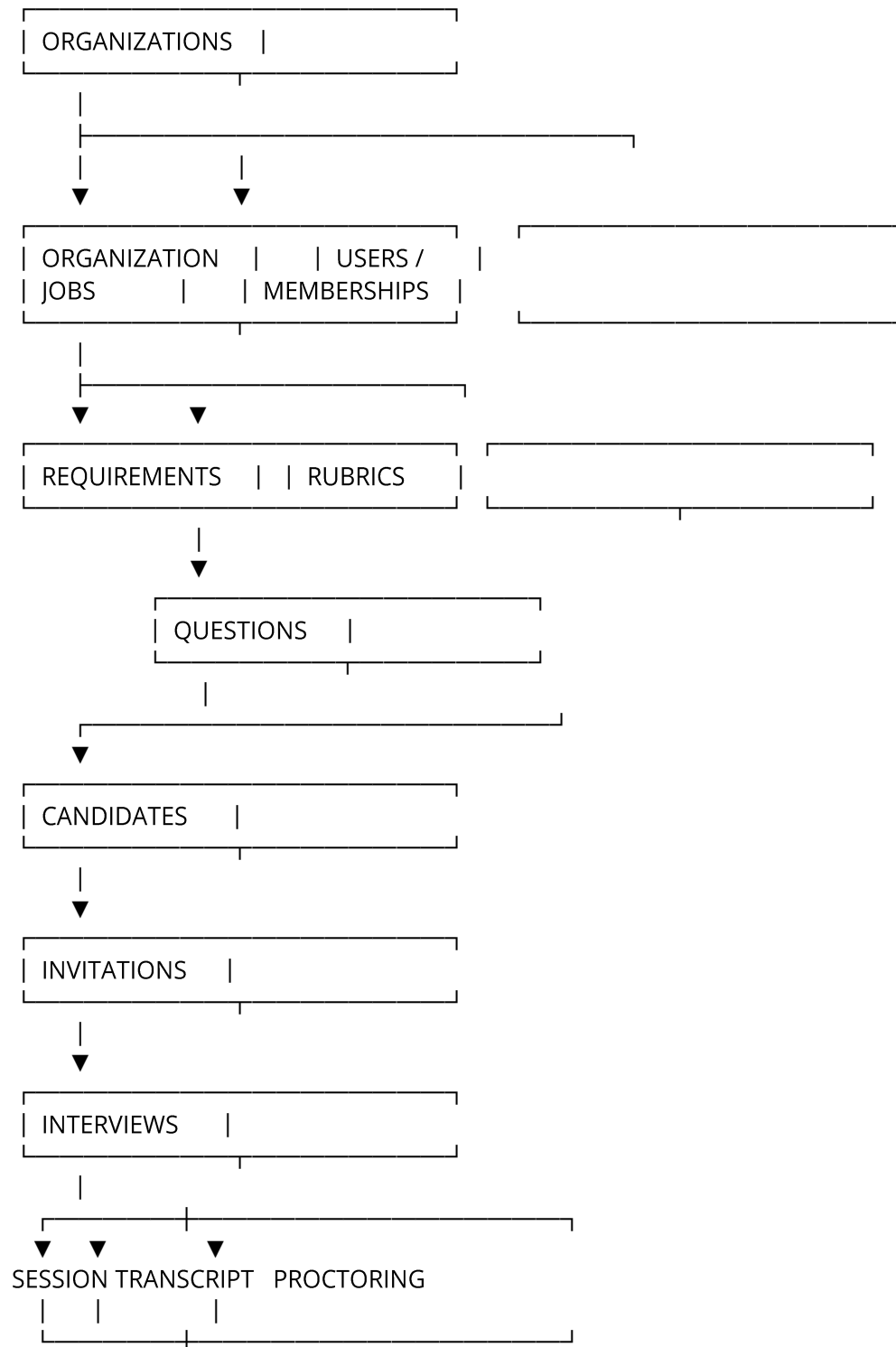


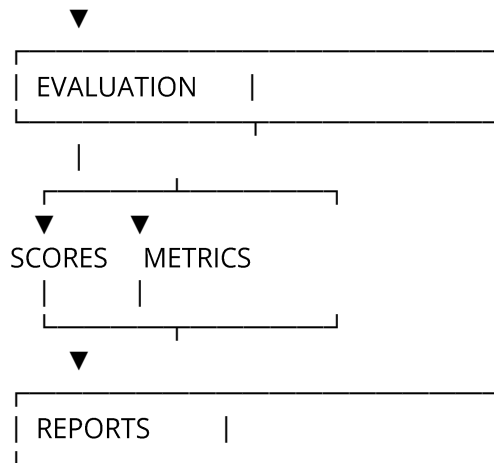
Possible analytical views include:

- Candidate count
- Interview completion
- Score distribution
- Candidate ranking
- Technical performance
- Communication metrics
- Interview status

32. Complete Entity Relationship Model

A simplified QualifyAI ER structure is:





33. Foreign-Key Strategy

Relationships should be enforced through database foreign keys wherever appropriate.

Examples:

```
jobs.organization_id
  ↓
organizations.id
interviews.job_id
  ↓
jobs.id
interviews.candidate_id
  ↓
candidates.id
evaluations.interview_id
  ↓
interviews.id
reports.interview_id
  ↓
interviews.id
```

This ensures that records cannot easily become orphaned.

34. Delete Strategy

Deletion behavior should be explicitly defined.

For dependent resources, the architecture should decide whether the correct behavior is:

CASCADE
RESTRICT
SET NULL
SOFT DELETE

For important assessment records such as interviews, transcripts, evaluations, and reports, **soft deletion or retention policies may be preferable to unrestricted destructive deletion**, depending on the final product requirements.

35. Auditability

Important assessment operations should be traceable.

Potential auditable actions include:

- Job creation
- Job modification
- Rubric modification
- Candidate invitation
- Interview creation
- Interview completion
- Evaluation generation
- Report generation
- Access-sensitive administrative actions

An audit mechanism may be expanded as the platform matures.

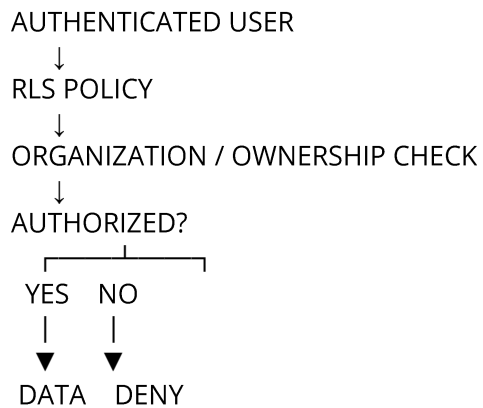
36. Row-Level Security Architecture

Where Supabase/PostgreSQL RLS is used, policies should enforce tenant and ownership boundaries.

The conceptual model is:

DATABASE REQUEST

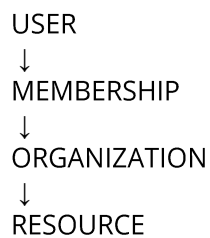




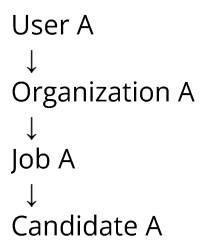
RLS should not be treated as optional protection for tenant-owned tables.

37. Organization-Level RLS

For organization-owned records:



Example:



A query attempting to retrieve:

Organization B → Job B

must be rejected unless the authenticated user has explicit authorization.

38. Candidate-Level RLS

Candidate access follows a different ownership dimension.

CANDIDATE USER
↓
CANDIDATE PROFILE
↓
AUTHORIZED INTERVIEW
↓
OWN TRANSCRIPT / FEEDBACK

The candidate must not be able to access another candidate's:

- Interview
- Transcript
- Score
- Report
- Proctoring data

39. Recruiter-Level Access

Recruiter access is organization-scoped.

RECRUITER
↓
ORGANIZATION MEMBERSHIP
↓
ORGANIZATION
↓
JOBS
↓
CANDIDATES
↓
INTERVIEWS
↓
EVALUATIONS / REPORTS

The recruiter does not receive unrestricted global database access.

40. RLS Policy Categories

Policies should be considered independently for:

SELECT
INSERT
UPDATE
DELETE

For example:

SELECT → Can this user view the row?
INSERT → Can this user create this row?
UPDATE → Can this user modify this row?
DELETE → Can this user remove this row?

This prevents accidentally allowing modification simply because read access exists.

41. Insert Security

Insert operations must validate ownership.

For example, a recruiter should not be able to create:

organization_id = Organization B

while authenticated as a member of Organization A.

The ownership relationship must be established through trusted authorization logic.

42. Update Security

Update policies must validate the existing record.

The security flow is:

EXISTING RECORD
↓

OWNERSHIP CHECK
↓
REQUESTED UPDATE
↓
AUTHORIZED?

A user must not be able to change an organization's ownership field to escape tenant restrictions.

43. Delete Security

Delete operations require especially strict controls.

DELETE REQUEST
↓
RESOURCE LOOKUP
↓
OWNERSHIP CHECK
↓
ROLE CHECK
↓
DELETE / DENY

Broad unrestricted delete conditions should never be used.

44. Backend + Database Security

RLS is one security layer, not the only one.

QualifyAI should use:

CLIENT
↓
AUTHENTICATION
↓
SERVER AUTHORIZATION
↓
BUSINESS VALIDATION
↓
DATABASE RLS
↓
DATABASE

This provides defense in depth.

45. AI Data Access Boundary

AI services should receive only the information required for their task.

For example:

```
INTERVIEW ENGINE
  ↓
Required Interview Context
  ↓
LLM
```

The AI provider should not receive unrelated organizational data.

The server remains responsible for:

- Selecting context
 - Removing unnecessary data
 - Validating responses
 - Persisting results
-

46. Transcript Security

Interview transcripts contain potentially sensitive candidate information.

Therefore:

```
TRANSCRIPT
  ↓
INTERVIEW
  ↓
CANDIDATE + ORGANIZATION
```

Access should be validated against the interview's ownership and authorized role.

A transcript URL or identifier alone must never grant access.

47. Report Security

Reports follow the same ownership chain:

REPORT
↓
INTERVIEW
↓
CANDIDATE
+
ORGANIZATION

Before returning a report, the system should verify:

REQUESTING USER
↓
AUTHORIZED ROLE
↓
CORRECT ORGANIZATION / CANDIDATE
↓
REPORT ACCESS

48. Data Integrity

The database should enforce integrity wherever possible through:

- Foreign keys
- Unique constraints
- Required fields
- Controlled status values
- Valid relationships
- Appropriate indexes
- Transaction boundaries

The application should not rely entirely on frontend validation.

49. Indexing Strategy

Indexes should prioritize common access patterns.

Likely indexed dimensions include:

organization_id
user_id
job_id
candidate_id
interview_id
status
created_at

Interview-related lookups should be optimized because the platform frequently queries:

Organization
→ Job
→ Candidate
→ Interview

Indexes should be introduced based on actual query patterns rather than indiscriminately indexing every field.

50. Transaction Boundaries

Operations involving multiple related records should use transactional boundaries where consistency is important.

Example:

```
CREATE INTERVIEW
↓
CREATE SESSION
↓
CREATE INITIAL STATE
```

If a critical step fails, the system should avoid leaving partially created records.

51. Data Lifecycle

A typical interview data lifecycle is:

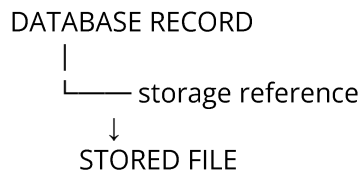


Retention duration should be defined according to the final product, legal, privacy, and organizational requirements.

52. Storage–Database Relationship

Files should not be treated as independent from their database metadata.

The conceptual relationship is:



Examples include:

- JD documents
- Generated reports
- Interview artifacts

The database should retain the metadata necessary to determine ownership and access.

53. Database Access Layer

The server should access PostgreSQL through the documented Prisma ORM layer.

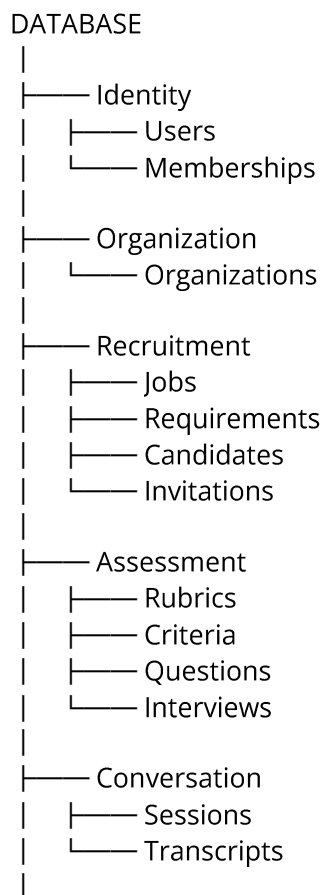
SERVER SERVICE
↓
PRISMA
↓
POSTGRESQL

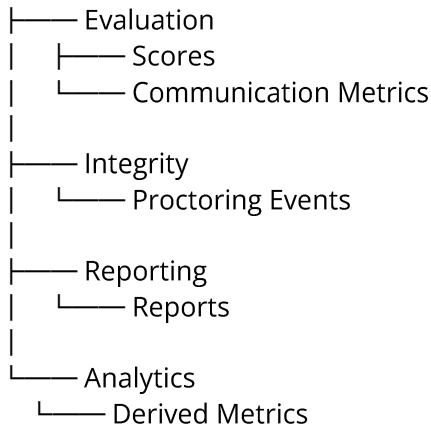
Application services should avoid scattering raw database logic throughout controllers.

The goal is:

CONTROLLER
↓
SERVICE
↓
DATA ACCESS
↓
DATABASE

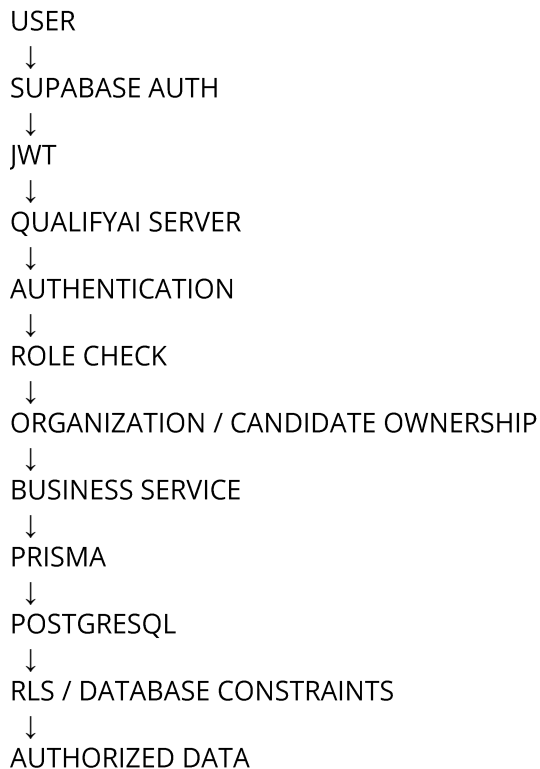
54. Database Architecture by Product Domain



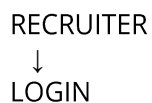


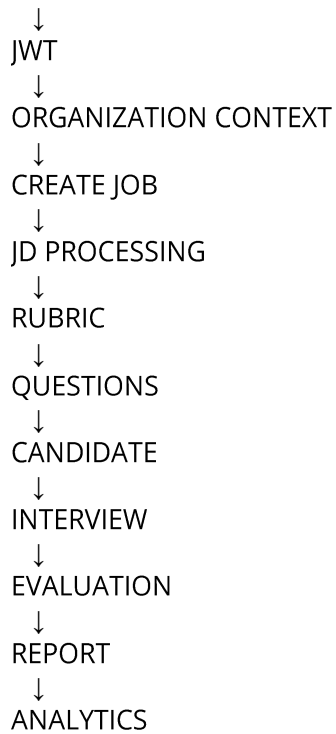
55. Complete Secure Data Flow

The complete secure data flow is:

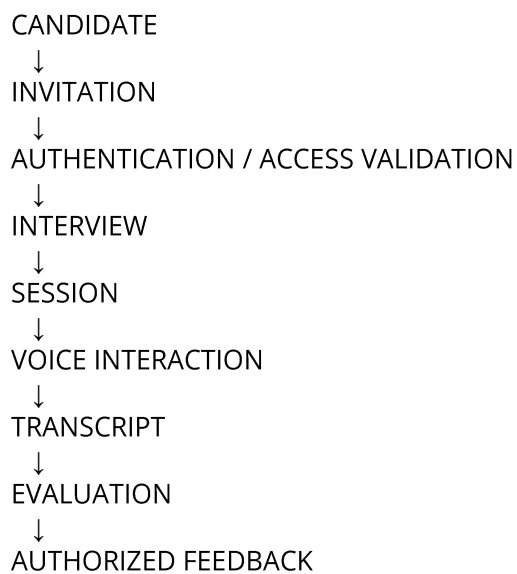


56. Recruiter Data Flow



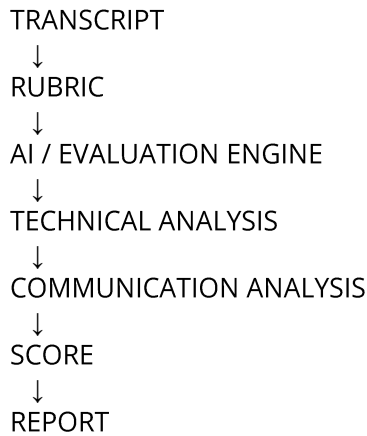


57. Candidate Data Flow



58. Evaluation Data Flow





The evaluation result should remain associated with the original interview and job context.

59. Proctoring Data Flow



The raw event record should remain distinguishable from the derived risk assessment.

60. Database Security Rules

The following rules are mandatory architectural principles:

Rule 1

Never rely only on frontend filtering for tenant security.

Rule 2

Every organization-owned resource must be traceable to its organization.

Rule 3

Candidates can access only their own authorized resources.

Rule 4

Recruiters can access only authorized organization resources.

Rule 5

AI providers must not receive unnecessary tenant data.

Rule 6

Database queries must use ownership-aware access patterns.

Rule 7

Insert, update, delete, and select authorization must be considered independently.

Rule 8

Sensitive interview information must never be exposed through predictable unrestricted identifiers.

Rule 9

Secrets must remain outside database records and frontend code unless explicitly required.

Rule 10

Database schema changes must remain synchronized with the architecture documentation.

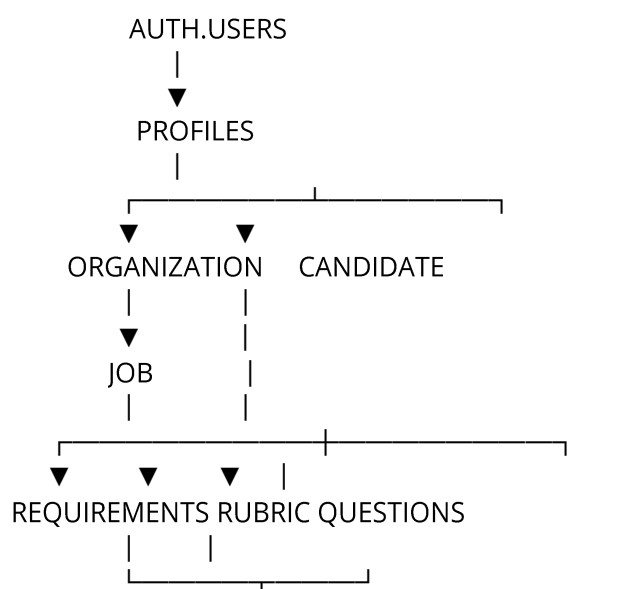
61. Database Acceptance Criteria

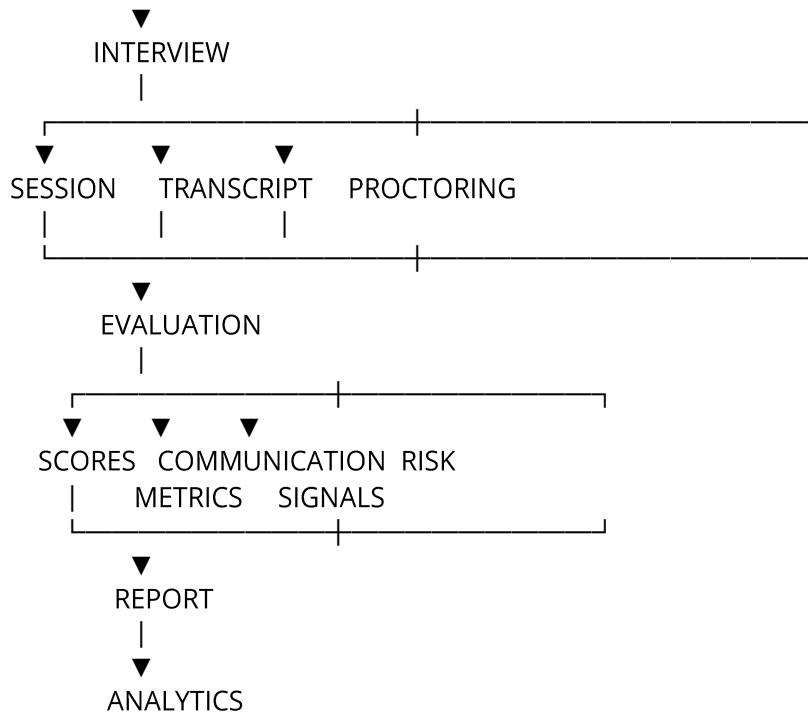
The database architecture is considered acceptable when:

- Users can be mapped securely to application profiles.
- Organizations are represented as tenant boundaries.
- Recruiters can access only authorized organizations.
- Candidates can access only their authorized interviews.
- Jobs are organization-scoped.
- Candidates are organization-scoped.
- Interviews are linked to jobs and candidates.
- Rubrics are associated with jobs/interviews.
- Questions are associated with the relevant assessment context.
- Transcripts are linked to interviews.
- Evaluations are linked to interviews.
- Communication metrics are linked to interviews.
- Proctoring events are linked to interviews.
- Reports are linked to interviews.
- Tenant isolation is enforced server-side.
- RLS is applied where required.
- Foreign-key relationships are defined.
- Sensitive data is not globally readable.
- AI-generated data passes through application validation.
- Database indexes support major access patterns.

62. Final Database Architecture

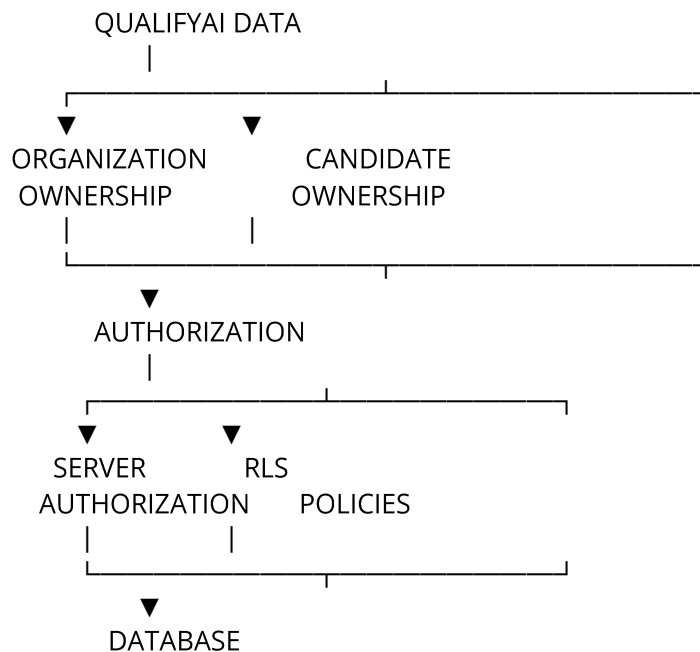
The complete QualifyAI database architecture can be summarized as:





63. Final Security Architecture

The database security model is:



The database therefore becomes an active security boundary rather than simply a passive data store.

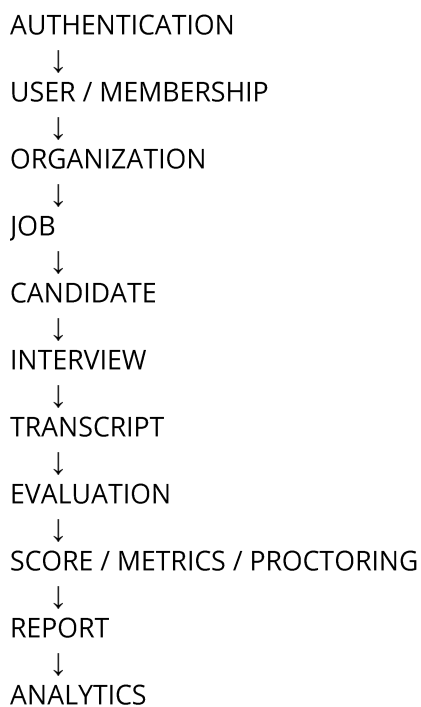
64. Final Definition

QualifyAI Database Architecture is a tenant-isolated PostgreSQL data architecture built around authenticated identities, organization ownership, job-specific recruitment data, interview-centric assessment records, evaluation results, communication metrics, proctoring events, and generated reports.

Its central principle is:

Every piece of sensitive recruitment and assessment data must have an explicit ownership path and an authorization boundary.

The architecture connects:



This provides QualifyAI with the database foundation required for a secure, multi-tenant technical recruitment and AI interview assessment platform.

Document 5 — Database Architecture & Security Model — Complete.